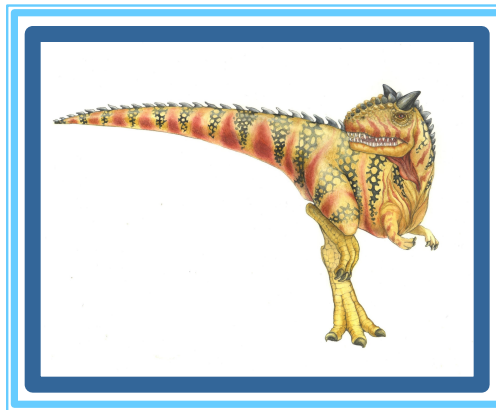


Process Scheduling

Rakhshan Imtiaz





First-Come, First-Served (FCFS)

Scheduling

- Simplest scheduling algorithm:
 - Run jobs in order that they arrive
- Non-Preemptive
 - A Process keeps CPU until done or I/O
- Advantage:
 - Simplicity

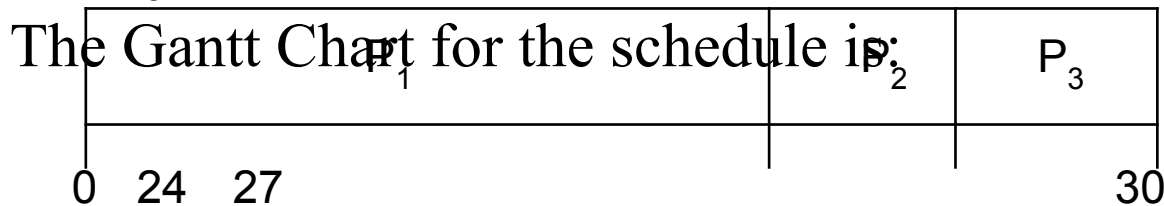




1- FCFS Scheduling

<u>Process</u>	<u>Burst Time</u>
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3

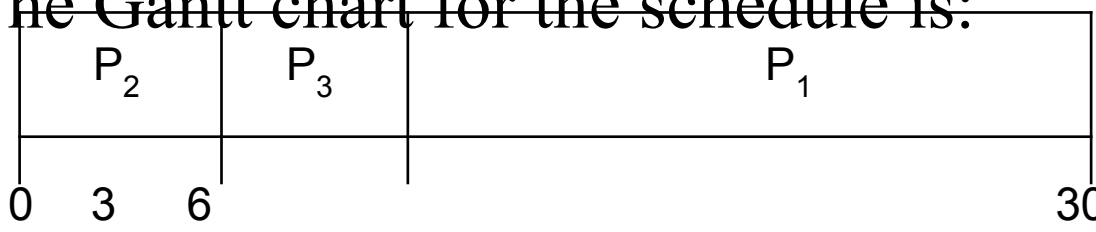


- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$





FCFS Scheduling (Cont.)

- Suppose that the processes arrive in the order P_2, P_3, P_1
- The Gantt chart for the schedule is:


The Gantt chart illustrates the execution of three processes: P_2 , P_3 , and P_1 . The timeline starts at 0. P_2 runs from 0 to 3. P_3 runs from 3 to 6. P_1 runs from 6 to 30. The total execution time is 30 units.
- Waiting time for $P_1 = 6; P_2 = 0; P_3 = 3$
- Average waiting time: $(6 + 0 + 3)/3 = 3$
- Much better than previous case

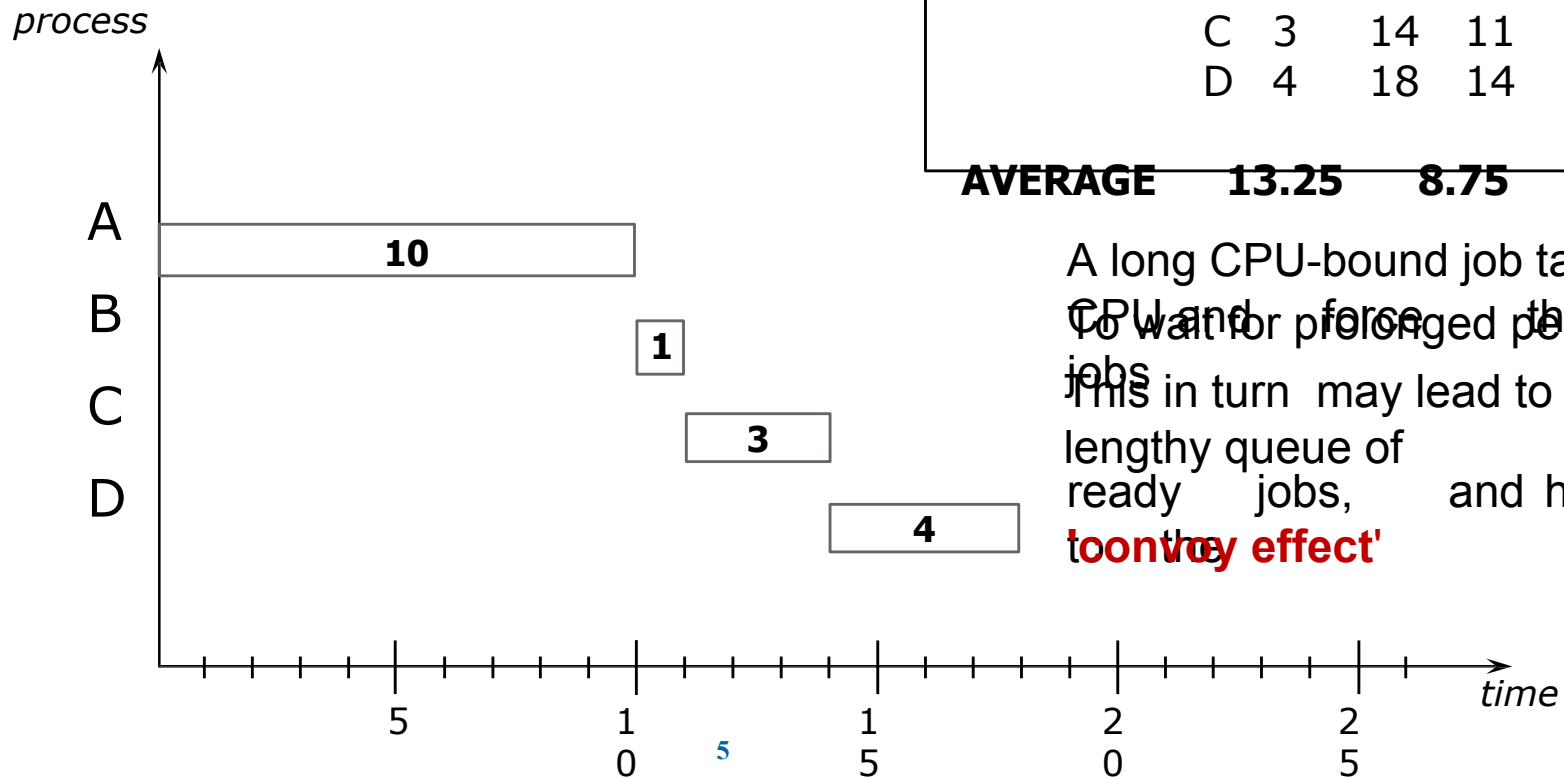




Convoy Effect

process	service time t_s	waiting time t_w	turnaround time t_t
A	10	10	0
B	1	11	10
C	3	14	11
D	4	18	14

AVERAGE 13.25 8.75



A long CPU-bound job takes the CPU and forces the shorter jobs to wait for prolonged periods. This in turn may lead to a lengthy queue of ready jobs, and hence to the **convoy effect**.





FCFS Scheduling

(Cont.)

- FCFS is:
 - Non-preemptive
 - Ready queue is a FIFO queue
 - Jobs arriving are placed at the end of ready queue
 - First job in ready queue runs to completion of CPU burst
- **Advantages:** simple, low overhead
- **Disadvantages:** long waiting time, inappropriate for interactive systems, large fluctuations in average turnaround time are possible





2. Shortest-Job-First (SJF)

Scheduling

- Associate with each process the length of its next CPU burst. Use these lengths to schedule the process with the shortest time
- Two schemes:
 - **Nonpreemptive** – once CPU is given to the process it cannot be preempted until the process completes its CPU burst
 - **Preemptive** – if a new process arrives with CPU burst length less than the remaining time of current executing process, preempt. This scheme is known as the **Shortest-Remaining- Time-First (SRTF)**
- SJF is optimal – gives minimum average waiting time for a given set of processes

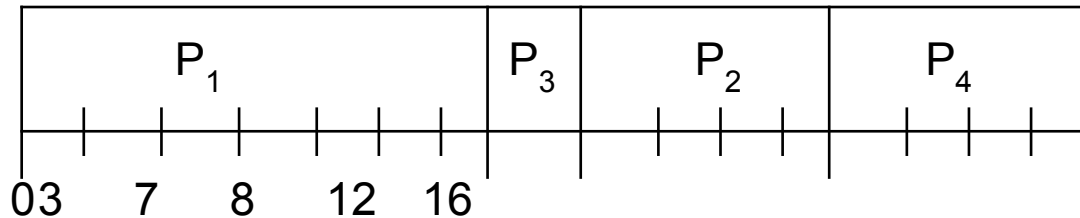




Example of Non-Preemptive SJF

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- SJF
(non-preemptive)



- Average waiting time = $(0 + 6 + 3 + 7)/4 = 4$





3- SRTF - Shortest Remaining Job First

- Preemptive version of SJF
- Ready queue ordered on length of time till completion (shortest first)
- Arriving jobs inserted at proper position
- shortest job
 - Runs to completion (i.e. CPU burst finishes) or
 - Runs until a job with a shorter remaining time arrives (i.e. placed in the ready queue)

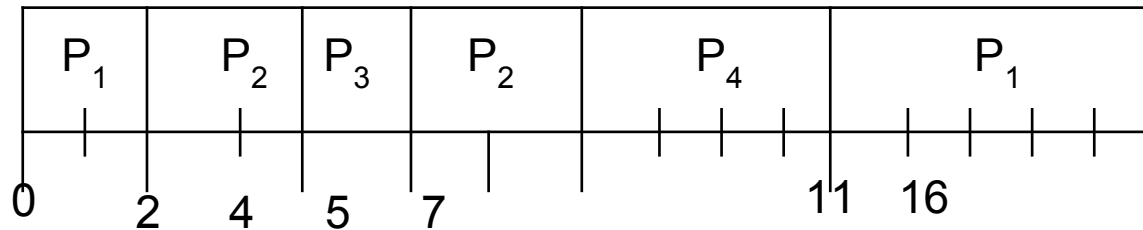




Example of Preemptive SJF (i.e., SRJF)

<u>Process</u>	<u>Arrival Time</u>	<u>Burst Time</u>
P_1	0.0	7
P_2	2.0	4
P_3	4.0	1
P_4	5.0	4

- Preemptive SJF (i.e., SRTF)



- Average waiting time = $(9 + 1 + 0 + 2)/4 = 3$





4. Priority

Scheduling

- A priority number (integer) is associated with each process
 - Priority can be internally computed (e.g., may involve time limits, memory usage) or externally (e.g., user type, funds being paid)
 - In SJF, priority is simply the predicted next CPU burst time
- The CPU is allocated to the process with the highest priority (smallest integer might mean highest priority)
- **Starvation** is a problem, where low priority processes may never execute
- **Solution:** as time progresses, the priority of the long waiting (starved) processes is increased. This is called **aging**





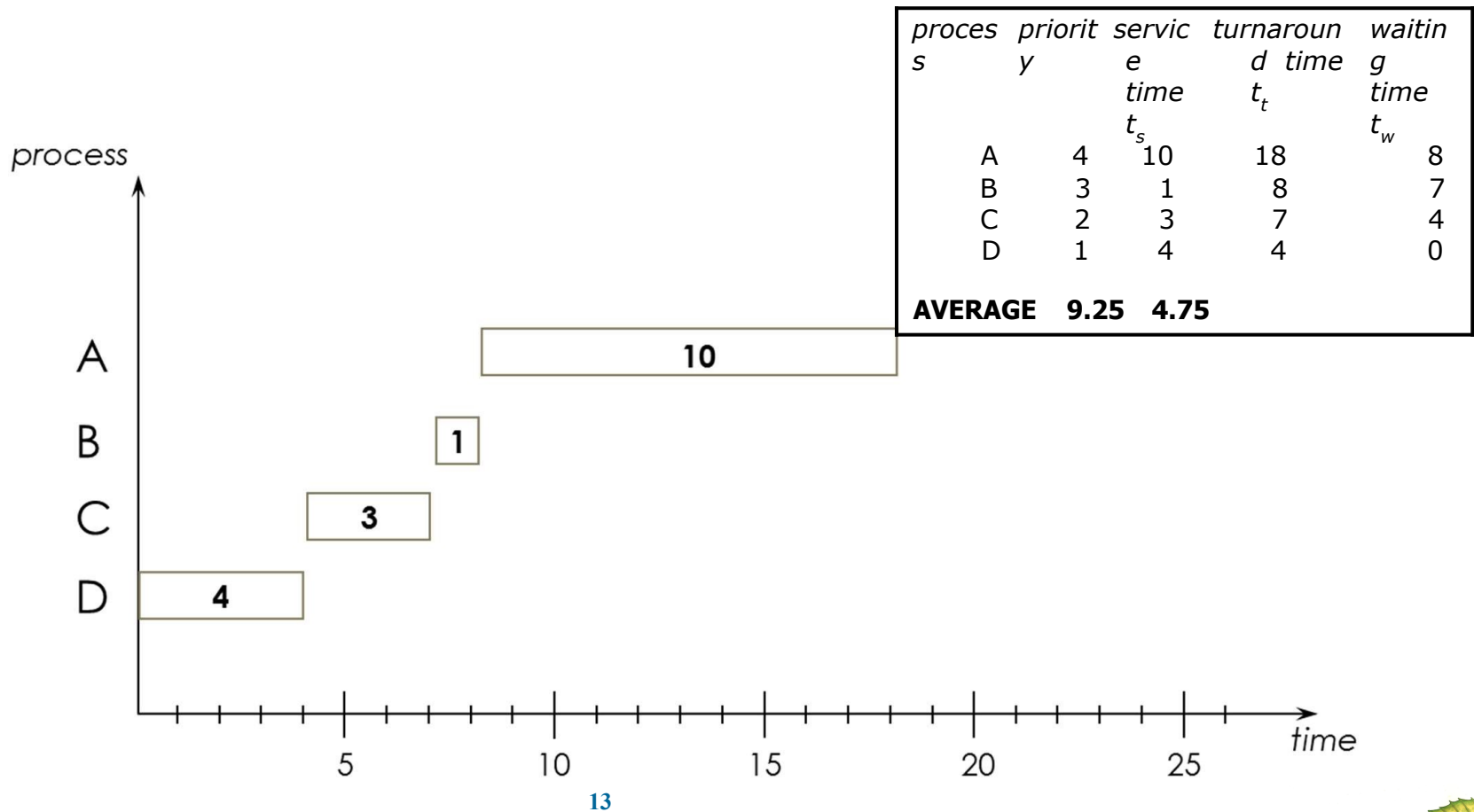
Priority scheduling

- Priority scheduling can be **Preemptive** or **Non- Preemptive**
- When a process arrives and enters the Ready Queue
- Its priority is compared with the currently Running Process
- If Higher
 - Preemptive Scheduling
 - Run the New process
 - Non-Preemptive Scheduling
 - Continue running the process





Priority Scheduling





Priority

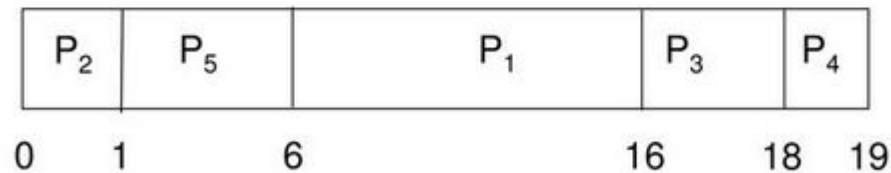
Scheduling

Priority Scheduling Example

Fixed (strict) priority scheduling

Process	Burst Time	Priority
P_1	10	3
P_2	1	1
P_3	2	4
P_4	1	5
P_5	5	2

* Priority scheduling Gantt Chart



* Average waiting time = 8.2 msec





5. Round Robin

(RR)

- RR reduces the penalty that short jobs suffer with FCFS by preempting running jobs periodically
- Each process gets a small unit of CPU time(*time quantum*), usually 10-100 milliseconds
- The CPU blocks the current job when its reserved *time quantum (time-slice)* is exhausted
 - The current job is then put at the end of the ready queue if it has not yet completed
 - If the current job is completed, it will exit the system (terminate)



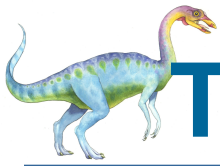


Round Robin

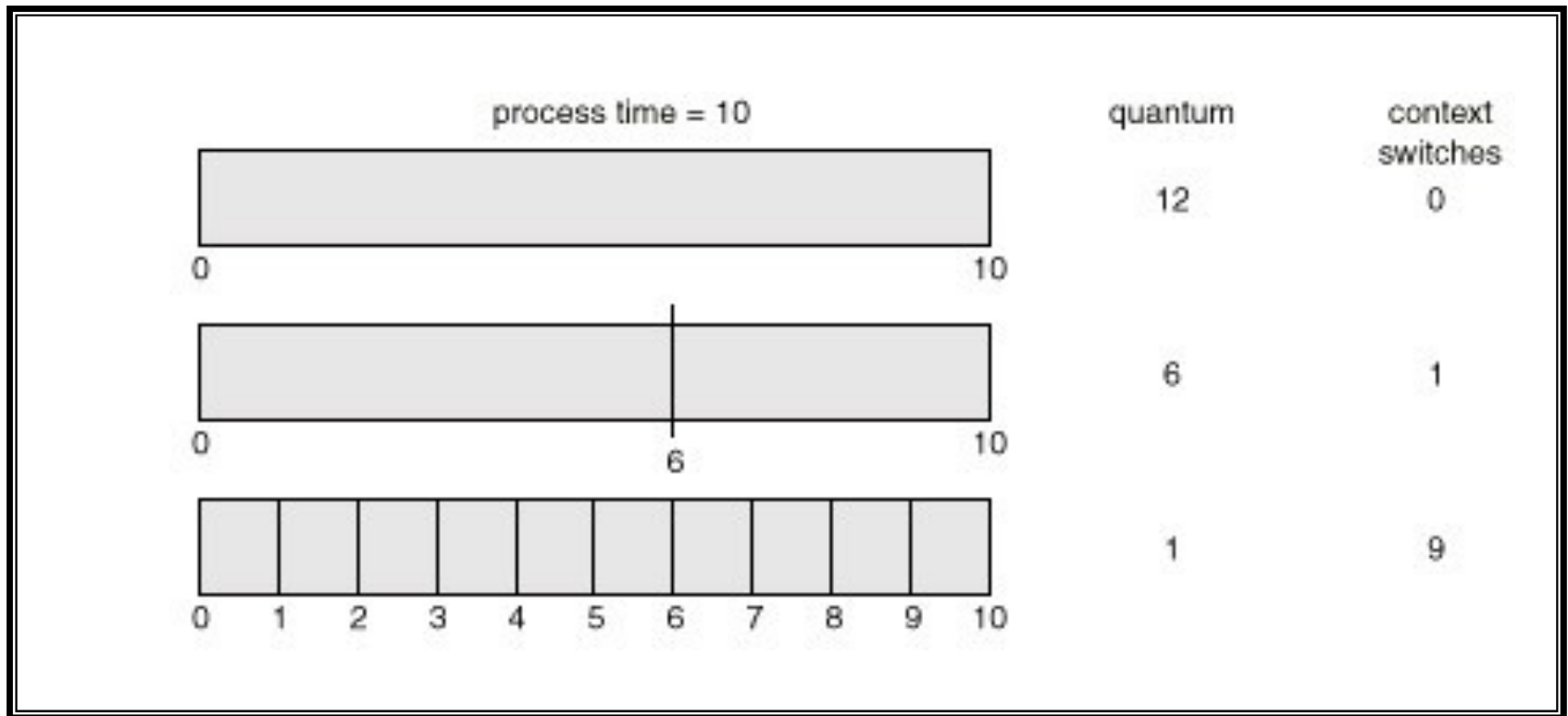
(RR)

- If there are n processes in the ready queue and the time quantum is q , then each process gets $1/n$ of the CPU time in chunks of at most q time units at once. No process waits more than $(n - 1)q$ time units
- Performance: the critical issue with the RR policy is the length of the quantum q
 - q is large: RR will behave like FIFO and hence interactive processes will suffer
 - q is small: the CPU will be spending more time on context switching
 - q must be large with respect to context switch, otherwise overhead is too high



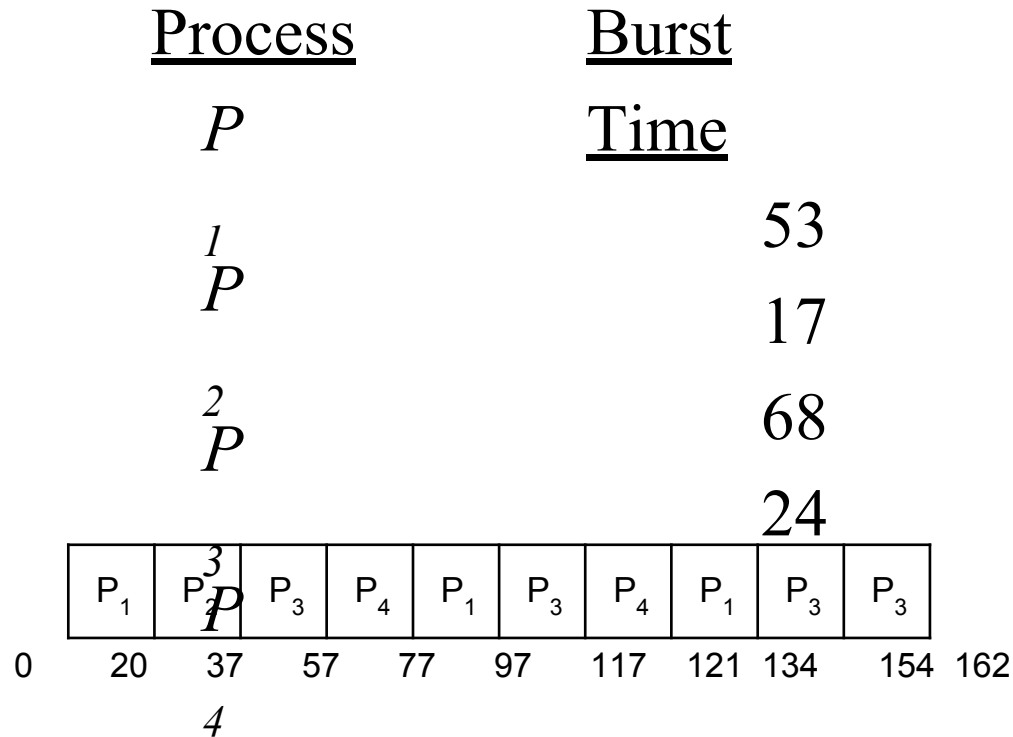


Time Quantum and Context Switch Time





Example of RR with Time Quantum = 20



- The Gantt chart is:
Typically, higher average turnaround than SJF
but better
response



END

